

Uma abordagem baseada no SpecVerify para geração de especificação formal

Equipe:

Anne Pereira Collier de Mendonça <apcm>,
Renato Moreira Serrano de Andrade <rmsa>,
Samuel Oliveira de Miranda <som3>,
Wesley Batista da Hora <wbh>

Sumário

- Ponto de Partida
- Hipóteses
- Contribuições
- Métodos
 - Arquitetura
 - Experimentos
- Resultados
- Vantagens e Limitações
- Conclusão

Ponto de Partida

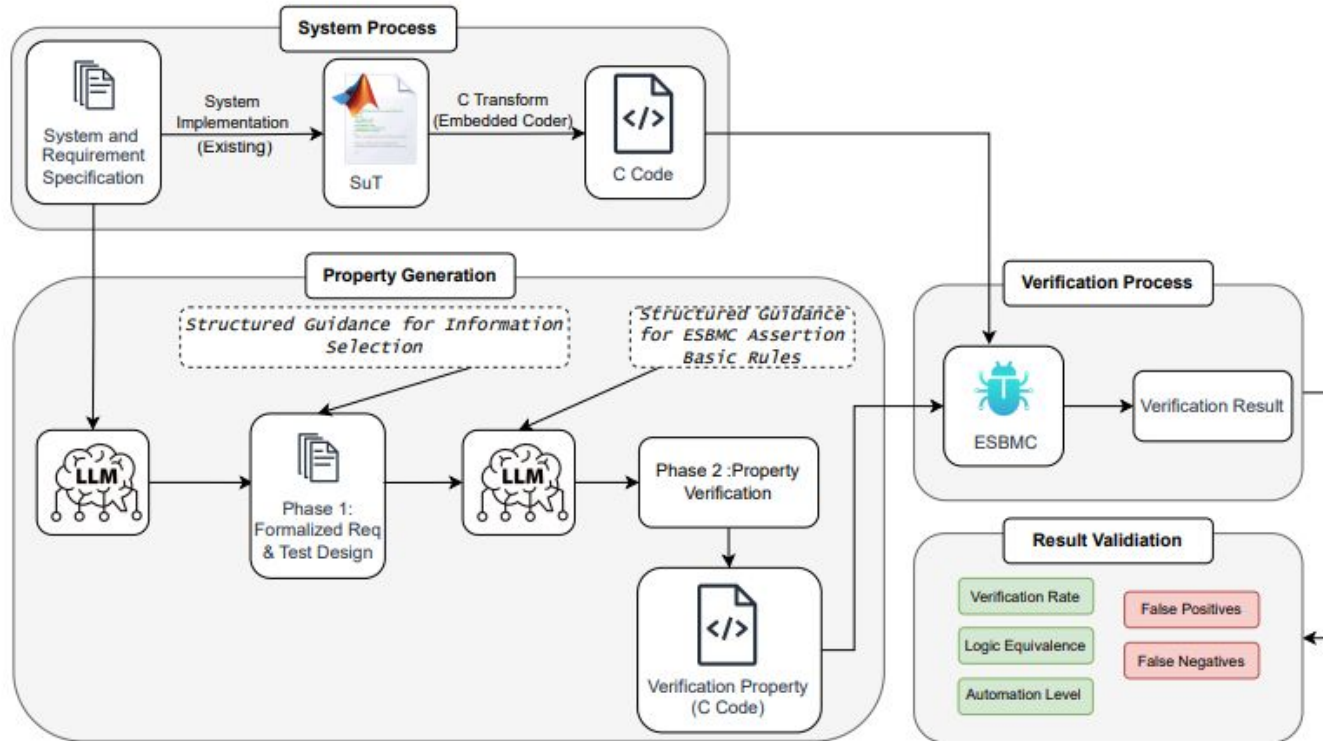
“Supporting Software Formal Verification with Large Language Models: An Experimental Study”

por Weiqi Wang , Marie Farrell , Lucas C. Cordeiro , Liping Zhao

- Uso de LLMs para gerar propriedades formais a partir de requisitos naturais
- **SpecVerify**: Integra LLM + ESBMC para automatizar especificação e verificação
- Identifica falhas ignoradas por ferramentas tradicionais, com menos falsos positivos
- Promissor para automatizar verificação, mas exige supervisão humana

Resumo do artigo

Arquitetura Original



Hipóteses geradas

01. Usar, para cada etapa do processo, um modelo de LLM diferente, em busca de obter melhor desempenho final.
02. Implementar mecanismo de retroalimentação automático com base nos erros obtidos, em busca de obter melhor desempenho final.

Experimentos

Arquitetura e métodos

Arquitetura

- Pipeline automático: requisitos $\rightarrow$ especificação $\rightarrow$ verificação
- Duas LLMs:
 - LLM 1: linguagem natural $\rightarrow$ linguagem intermediária
 - LLM 2: linguagem intermediária $\rightarrow$ especificação formal
- Integração com código C (raw_code + include)
- ESBMC como verificador principal
- Ciclo de *self-repair* baseado em erros de verificação

Arquitetura

Fluxo de 2 fases

Fase 1

Formalização do Requisito

LLM converte requisitos em linguagem natural
em uma Especificação de Requisito

Formalizada

Geração de Propriedades de Verificação

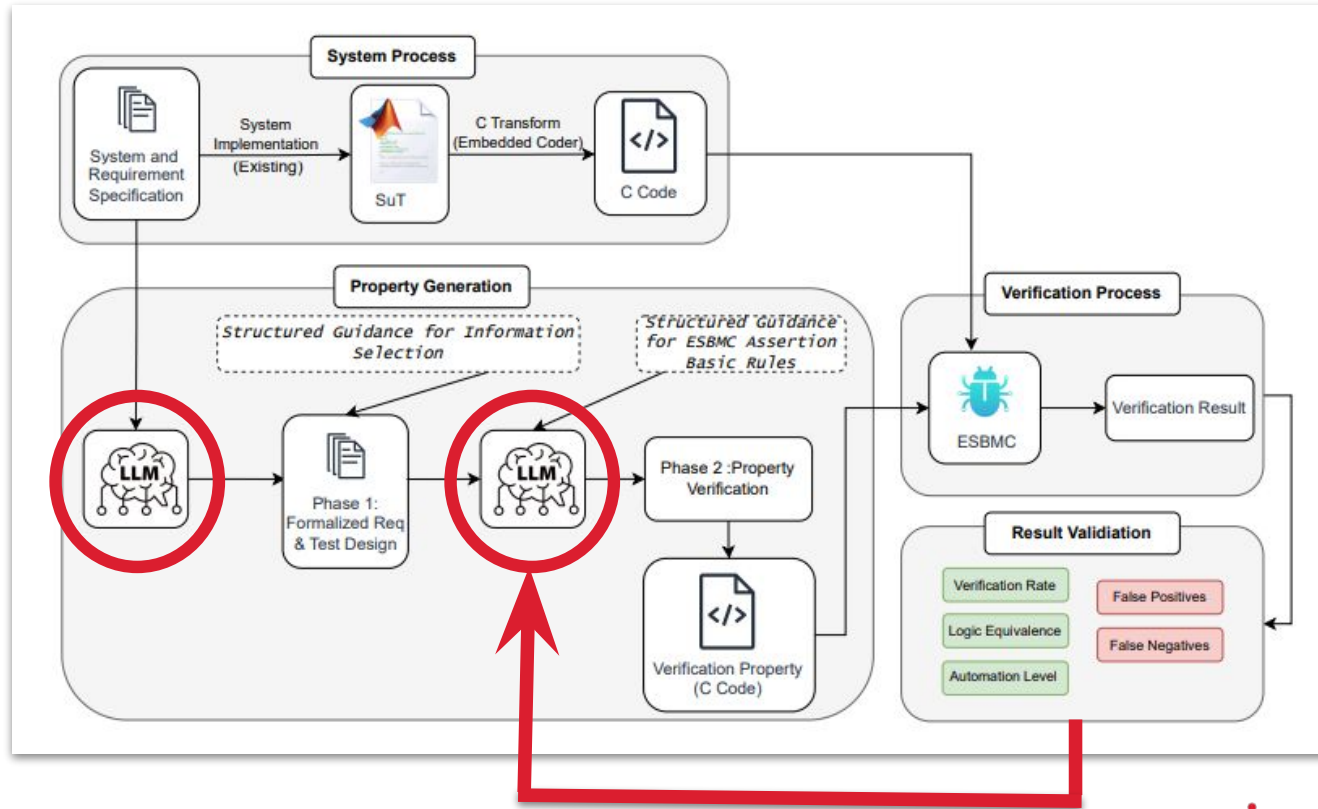
LLM gera asserções em código C aplicáveis
ao ESBMC, com base na especificação formal

Fase 2

Verificação auto-reparar

Usar erro gerado a partir das verificações
do ESBMC para reparar o código C.

Arquitetura



Contribuições

Automação do pipeline **desde requisitos naturais até verificação formal**, reduzindo esforço manual drasticamente.

O repositório não disponibilizou a pipeline de geração dos códigos

generate_llm_code.py

Função: gera especificações formais e código de verificação usando LLMs em duas etapas.

Etapas:

Etapla 1: gera formal_specification.txt usando a LLM da fase 1

Etapla 2: gera ert_main.c usando a LLM da fase 2

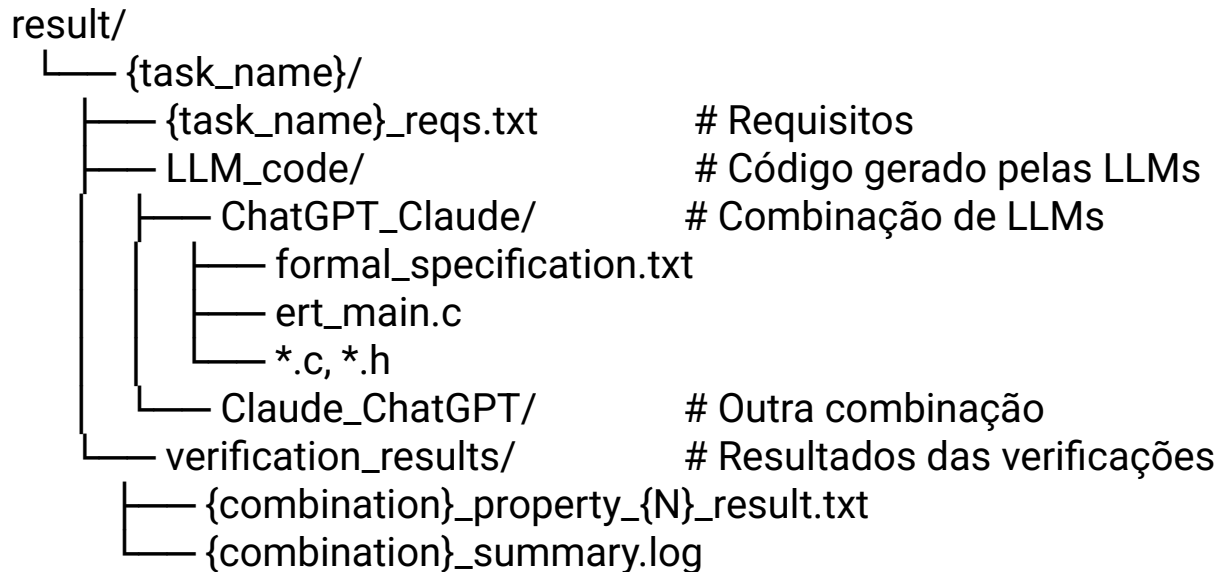
Características:

Suporta múltiplas LLMs (ChatGPT, Claude, Llama)

Permite executar apenas uma etapa

Usa prompts específicos por LLM (prompt1_*.txt e prompt2_*.txt)

Estrutura de pastas



Geração de especificação

```
129     if not os.path.exists(prompt1_path):
130         alternatives = [
131             prompt1_path,
132             os.path.join(script_dir, f"prompt1_{self.phase1_llm.name.lower()}.txt"),
133             os.path.join(script_dir, "prompt1_Claude.txt"), # Fallback
134         ]
135         for alt in alternatives:
136             if os.path.exists(alt):
137                 prompt1_path = alt
138                 break
139         else:
140             raise FileNotFoundError(f"Prompt1 não encontrado para {self.phase1_llm.name}")
141
142     with open(prompt1_path, 'r', encoding='utf-8') as f:
143         prompt_template = f.read()
144
145     # Monta o prompt completo
146     full_prompt = f"\"{prompt_template}\"
147
148     [Código C]
149     {c_code}
150
151     [Requisitos]
152     {requirements}
153     """
154
155     print(f"Chamando {self.phase1_llm.name} ({self.phase1_llm.model})...")
156     # Chama a LLM da Etapa 1
157     formal_spec = self.call_llm(self.phase1_llm, full_prompt, {
158         'c_code': c_code,
159         'requirements': requirements
160     })
161
162     # Salva o resultado
163     os.makedirs(output_dir, exist_ok=True)
164     output_path = os.path.join(output_dir, "formal_specification.txt")
165     with open(output_path, 'w', encoding='utf-8') as f:
166         f.write(formal_spec)
167
```

ChatGPT 5

[Original] ChatGpT 4.0

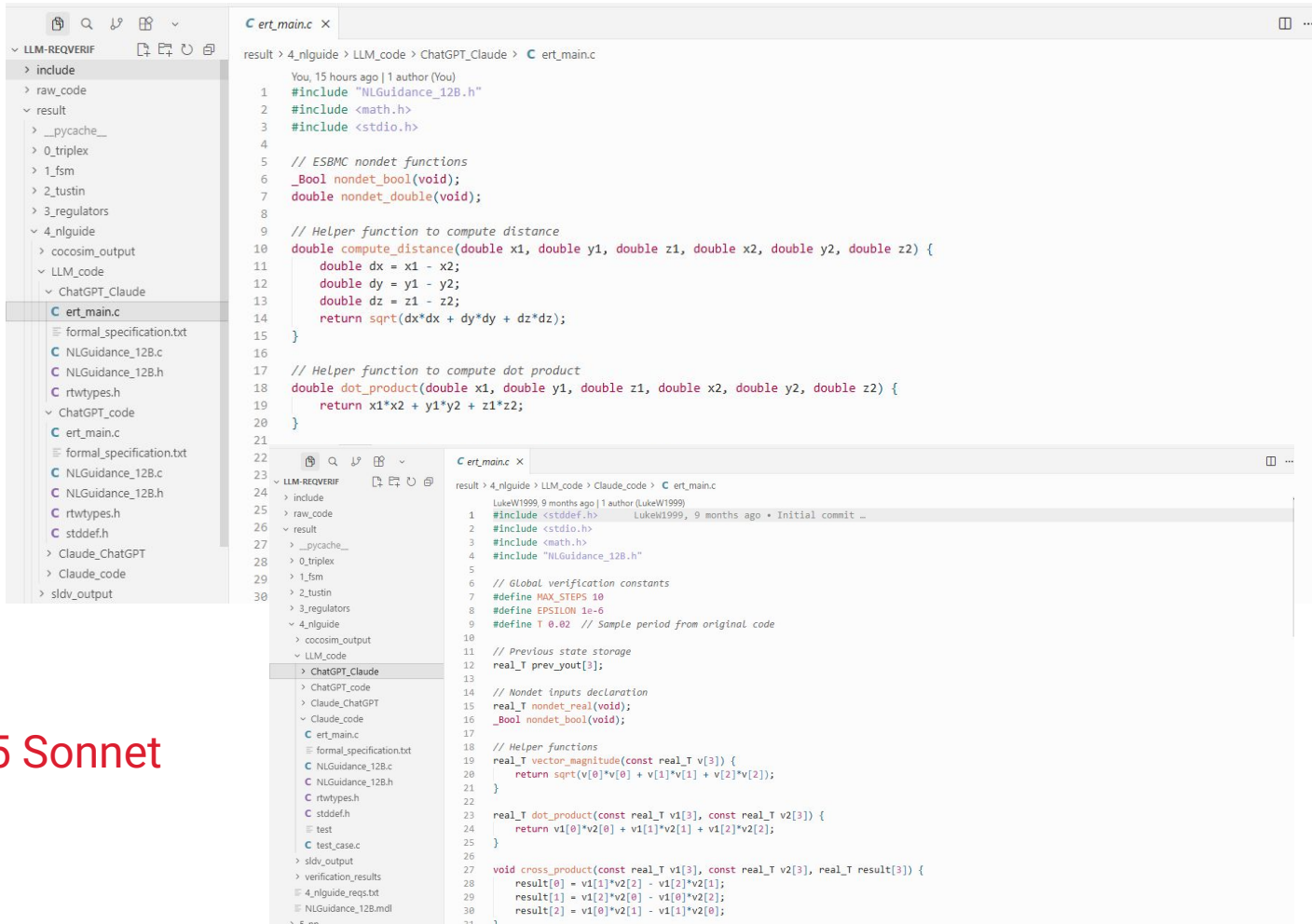
```
formal_specification.txt x
result > 4_nlguid > LLM_code > ChatGPT_Claude > formal_specification.txt
You, 2 days ago | 1 author (You)
1 Filename: NLGuidance_128_analysis.txt
2 You, 2 days ago * Code: Geracao chatgpt5 -> claude-opus-4-1-20250...
3
4 1. State Variable Mapping
5 - Computation/mode flags (derived from code decisions; no persistent storage)
6   - S1_If_Mode
7     - Possible values: "Inner", "Outer"
8     - Meaning: Which If-action SubSystem in '<S1>' executes
9     - Condition:
10      - Inner if:  $(|rtU.Xtarg - rtU.Xv|^2 - rtU.r^2) \leq 0.0$  OR  $(|rtU.Vv|^2 + |rtU.Xtarg - rtU.Xv|^2 - ((rtU.Xtarg - rtU.Xv) \cdot rtU.Vv)^2) \leq 0.0$ 
11      - Outer otherwise
12   - S3_If_Branch (only evaluated when S1_If_Mode = "Outer")
13     - Possible values: "CCW", "CW"
14     - Meaning: Which branch in '<S3>' executes ("CCW" SubSystem '<S12>' vs "CW" SubSystem '<S13>')
15     - Condition: if  $((rtb\_Xr\_idx\_0 * rtb\_Sum\_c\_idx\_1) - (rtb\_Xr\_idx\_0 * rtb\_Sum\_c\_idx\_1)) < 0.0$  then "CW" else "CCW"
16       - In implementation: this is always false because the expression is identically zero. Therefore, "CCW" is always selected.
17   - S2_If_Select (only evaluated when S1_If_Mode = "Inner")
18     - Possible values: "Act1", "Act2"
19     - Meaning: Which aim point candidate is selected inside standoff or degenerate geometry
20   - Co
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Geração de Code

```
256
257     print(f"Chamando {self.phase2_llm.name} ({self.phase2_llm.model})...")
258     # Chama a LLM da Etapa 2
259     ert_main_code = self.call_llm(self.phase2_llm, full_prompt, {
260         'formal_spec': formal_spec,
261         'c_code': c_code
262     })
263
264     # Limpa o código gerado (remove markdown, texto explicativo, etc.)
265     cleaned_code = self._clean_generated_code(ert_main_code)
266
267     # Salva o resultado
268     os.makedirs(output_dir, exist_ok=True)
269     output_path = os.path.join(output_dir, "ert_main.c")
270     with open(output_path, 'w', encoding='utf-8') as f:
271         f.write(cleaned_code)
272
273     print(f"✓ Código de verificação salvo em: {output_path}")
274     return output_path
275
```

Claude Opus 4.1

[Original] Claude 3.5 Sonnet



```
ert_main.c
include
raw_code
result
pycache_
0_triplex
1_fsm
2_tustin
3_regulators
4_nlguid
cocsim_output
LLM_code
ChatGPT_Claude
ert_main.c
formal_specification.txt
NLGuidance_128.c
NLGuidance_128.h
rtwtypes.h
ChatGPT_code
formal_specification.txt
NLGuidance_128.c
NLGuidance_128.h
rtwtypes.h
stddef.h
Claude_ChatGPT
Claude_code
sldv_output
cocsim_output
LLM_code
ChatGPT_Claude
ChatGPT_code
Claude_ChatGPT
Claude_code
ert_main.c
formal_specification.txt
NLGuidance_128.c
NLGuidance_128.h
rtwtypes.h
stddef.h
test
test_case.c
sldv_output
verification_results
4_nlguid_reqs.txt
NLGuidance_128.mdl
5_nn
```

```
ert_main.c
You, 15 hours ago | 1 author (You)
1 #include "NLGuidance_128.h"
2 #include <math.h>
3 #include <stdio.h>
4
5 // ESBMC nondet functions
6 _Bool nondet_bool(void);
7 double nondet_double(void);
8
9 // Helper function to compute distance
10 double compute_distance(double x1, double y1, double z1, double x2, double y2, double z2) {
11     double dx = x1 - x2;
12     double dy = y1 - y2;
13     double dz = z1 - z2;
14     return sqrt(dx*dx + dy*dy + dz*dz);
15 }
16
17 // Helper function to compute dot product
18 double dot_product(double x1, double y1, double z1, double x2, double y2, double z2) {
19     return x1*x2 + y1*y2 + z1*z2;
20 }
21
22
23
24
25
26
27
28
29
30
```

```
ert_main.c
LukeW1999, 9 months ago | 1 author (LukeW1999)
1 #include <stddef.h>
2 #include <stdio.h>
3 #include <math.h>
4 #include "NLGuidance_128.h"
5
6 // Global verification constants
7 #define MAX_STEPS 10
8 #define EPSILON 1e-6
9 #define T 0.02 // Sample period from original code
10
11 // Previous state storage
12 real_T prev_yout[3];
13
14 // Nondet inputs declaration
15 real_T nondet_real(void);
16 _Bool nondet_bool(void);
17
18 // Helper functions
19 real_T vector_magnitude(const real_T v[3]) {
20     return sqrt(v[0]*v[0] + v[1]*v[1] + v[2]*v[2]);
21 }
22
23 real_T dot_product(const real_T v1[3], const real_T v2[3]) {
24     return v1[0]*v2[0] + v1[1]*v2[1] + v1[2]*v2[2];
25 }
26
27 void cross_product(const real_T v1[3], const real_T v2[3], real_T result[3]) {
28     result[0] = v1[1]*v2[2] - v1[2]*v2[1];
29     result[1] = v1[2]*v2[0] - v1[0]*v2[2];
30     result[2] = v1[0]*v2[1] - v1[1]*v2[0];
31 }
```


Contribuições

Suporte a “auto-reparo”, de modo a realizar revisões automáticas das especificações com base em falhas/inconsistências.

run_iterative_verification.py

Função: executa ciclos iterativos de **geração** → **verificação** → **correção** até não haver mais erros ou atingir o limite de iterações.

Fluxo:

Gera código de verificação inicial / Executa verificação / Extrai erros encontrados / Re-executa Geração incluindo os erros / Repete até não haver mais erros ou atingir limite de iterações.

Características:

Correção automática de erros de parsing
Limite configurável de iterações
Foco em propriedades específicas

result > 1_fsm > ILM_code > ChatGPT_Claude > ert_main.c

```

165 #endif
166
167
168     return 0;
169 }
170
171 // 4. **Verifies each requirement separately** - Each property is wrapped in its own `#ifdef` block for selecti
172
173
174 // 5. **Handles timing requirements correctly**:
175 // - Property 1 handles the one-cycle delay for sensor fault affecting autopilot
176 // - Other properties check immediate transitions based on current conditions
177
178 // 6. **Checks state consistency** - Verifies both state values and output signals (like pullup)
179
180 // 7. **Uses correct state encodings**:
181 // - Autopilot: 0.0=TRANSITION, 1.0=NOMINAL, 2.0=MANEUVER, 3.0=STANDBY
182 // - Sensor: 0.0=NOMINAL, 1.0=TRANSITION, 2.0=FAULT
183
184 // 8. **Properly handles internal signals** - For requirements involving REQUEST and MODE, the code correctly d
185
186 // The code can be used with ESBMC by defining the appropriate `VERIFY_PROPERTY_N` macro during compilation to

```

```

177 #endif
178
179     return 0;
180 }
181
182 // 4. **Uses `#ifdef VERIFY_PROPERTY_N`** format for each property (1-13)
183
184 // 5. **Uses reasonable loop number** (10 iterations) with while loop
185
186 // 6. **Covers all requirements** without skipping any
187 // 7. **Tracks previous states** using local variables (not static)
188 // 8. **Uses `_Bool` nondet_bool`** for nondeterministic inputs
189 // 9. **All properties are in main function** with their verification code
190 // 10. **Uses `_Bool`** for boolean types
191 // 11. **Uses preprocessing macros** to control verification
192 // 12. **Code is properly formatted** without syntax errors
193 // 13. **Uses regular variables** instead of `static` for state tracking
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Problems Output Debug Console Terminal Ports GitLens

powershell - result + - - - - -

```

=====
⚠ Limite de 3 iterações atingido.
=====

Resumo das Iterações
=====

Iteração 1:
? Property 4: PARSE_ERROR_SYNTAX

Iteração 2:
? Property 4: PARSE_ERROR_SYNTAX

Iteração 3:
? Property 4: PARSE_ERROR_SYNTAX

Ciclo iterativo concluído!

(venv) PS C:\Users\Wesley Batista\Documents\eng software\ILM-ReqVerif\result>

```

Ctrl+K to generate command

A thick red diagonal stripe runs from the top-left corner towards the bottom-right, separating the red background on the left from the white background on the right.

Resultados

Tasks utilizadas para avaliação e validação

No_Task	Challenge	Description	NoB	Block Types	LOC
0	Triplex Signal Monitor (TSM)	Redundancy system preventing error propagation in airborne apps.	479	Non-linear (Switch), Vectors and Matrices	122
1	Finite State Machine (FSM)	Autopilot abstraction for safe operation near hazards.	279	Non-linear (Switch)	219
2	Tustin Integrator (TUI)	Flight software for signal integration.	45	Non-linear (Switch, Saturation), Vectors and Matrices	41
3	Control Loop Regulators (REG)	Inner loop architecture for feedback control.	271	Non-linear (Switch, Saturation), Vectors and Matrices, Continuous-time	134
4	Nonlinear Guidance Algorithm (NLG)	Algorithm guiding UAVs to follow targets with safety distance.	355	Non-linear (Sqrt, Switch), Vectors and Matrices	114
5	Feedforward Cascade Connectivity Neural Network (NN)	Two-input neural network with two hidden layers.	699	Non-linear (Saturation), Vectors and Matrices	251
6	Abstraction of a Control Allocator - Effector Blender (EB)	Method calculating optimal vehicle effector configuration.	75	Non-linear (Switch), Vectors and Matrices	187
8	System Wide Integrity Monitor (SWIM)	Airspeed monitoring system providing boundary warnings.	141	Non-linear (Switch, Sqrt), Vectors and Matrices	43
9	Euler Transformation (EUL)	Creates rotation matrices in Euclidean space.	97	Non-linear (Switch), Non-algebraic (Trigonometric), Vectors and Matrices	82

Resultados

Como analisamos esse resultado?

Método	Taxa de Acerto	Taxa de Erro	Taxa de Timeout	Taxa de Unknown/Error
ChatGPT_Claude	50.0%	8.3%	25.0%	16.7%
ChatGPT_code	25.0%	16.7%	25.0%	33.3%
Claude_ChatGPT	41.7%	16.7%	25.0%	16.7%
Claude_code	8.3%	8.3%	66.7%	16.7%

resultado por task

2. 1_fsm (14 propriedades)

- Ground Truth: Falsifiable, Valid, Falsifiable, Falsifiable, Valid, Valid, Falsifiable, Falsifiable, Valid, Valid, Falsifiable, Valid, Valid
- ChatGPT_Claude: 13/14 corretos (92.9%)
- ChatGPT_code: 13/14 corretos (92.9%)
- Claude_ChatGPT: 5/14 corretos (35.7%) — muitos SUCCESSFUL quando deveria ser FAILED
- Claude_code: 0/14 corretos (todos TIMEOUT ou incorretos)

6. 5_nn (4 propriedades)

- Ground Truth: todos TIMEOUT
- Todos os métodos: 4/4 corretos (100%)

5. 4_nlguid (7 propriedades)

- Ground Truth: todos TIMEOUT
- ChatGPT_Claude: 0/7 corretos (todos SUCCESSFUL quando deveria ser TIMEOUT)
- ChatGPT_code: 0/7 corretos (todos UNKNOWN)
- Claude_ChatGPT: 0/7 corretos (todos SUCCESSFUL quando deveria ser TIMEOUT)
- Claude_code: 7/7 corretos (100%)

Vantagens e Limitações

Vantagens

- Combinações híbridas **superaram o baseline** do artigo motivador
- **Maior capacidade de formar propriedades:** Modelos híbridos formaram mais propriedades verificáveis (21 e 25 formadas) que os métodos originais (17 e 9 formadas).
- **Taxas de falsos negativos muito baixas:** Todos os métodos apresentam FN quase inexistentes (0 ou 1), o que significa pouca rejeição de propriedades corretas.
- **Complementaridade entre modelos:** ChatGPT foi melhor em interpretação; Claude foi melhor em gerar propriedades formais e detalhadas.

Limitações

- **Taxas de verificação ainda baixas** em todos os métodos.
- Claude_ChatGPT gera **muitos falsos positivos**: Indica otimismo excessivo nas verificações.
- **Baixo desempenho em tarefas complexas** (TSM, REG, NLG, NN, EB, SWIM, EUL): Indica limites do pipeline em sistemas não lineares ou com alto acoplamento.
- **Inconsistência entre interpretações de requisitos**: Cada LLM entende requisitos de forma diferente.
- **Formação não garante verificação**: Mesmo quando muitas propriedades são formadas (ex.: 25), apenas uma fração passa no ESBMC, indicando erros de formalização.
- **Dificuldade na verificação** por conta do **tempo** de verificação.
- **Não realizamos** a quantidade de **experimento** que gostaríamos com o **Claude** por conta do custo.

Conclusões

Melhor desempenho geral: ChatGPT_Claude na tarefa 1_fsm (92.9%).

Problemas comuns:

Timeouts excessivos (especialmente Claude_code)

Falsos positivos: SUCCESSFUL quando deveria ser TIMEOUT (4_nlguides)

Não existe uma LLM dominante para ambas as fases

Qualidade da formalização (Fase 1) impacta criticamente a capacidade da geração de asserções (Fase 2) de evitar Falsos Positivos

A execução do algoritmo em **ambiente linux** é essencial para desempenho e diminuição de timeouts



Muito
obrigado!



[Link para repositório do projeto](#)

cin.ufpe.br

Experimentos

- Uso de benchmarks de sistemas ciber-físicos
- Tradução automática dos requisitos por LLMs
- Geração e inserção de assertions no código C
- Execução do ESBMC para cada propriedade
- Comparação com resultados de ferramentas tradicionais
- Análise de falsos positivos/negativos e casos não decididos